

REKOLT Produce Tracker - Design Document (Objective 4)

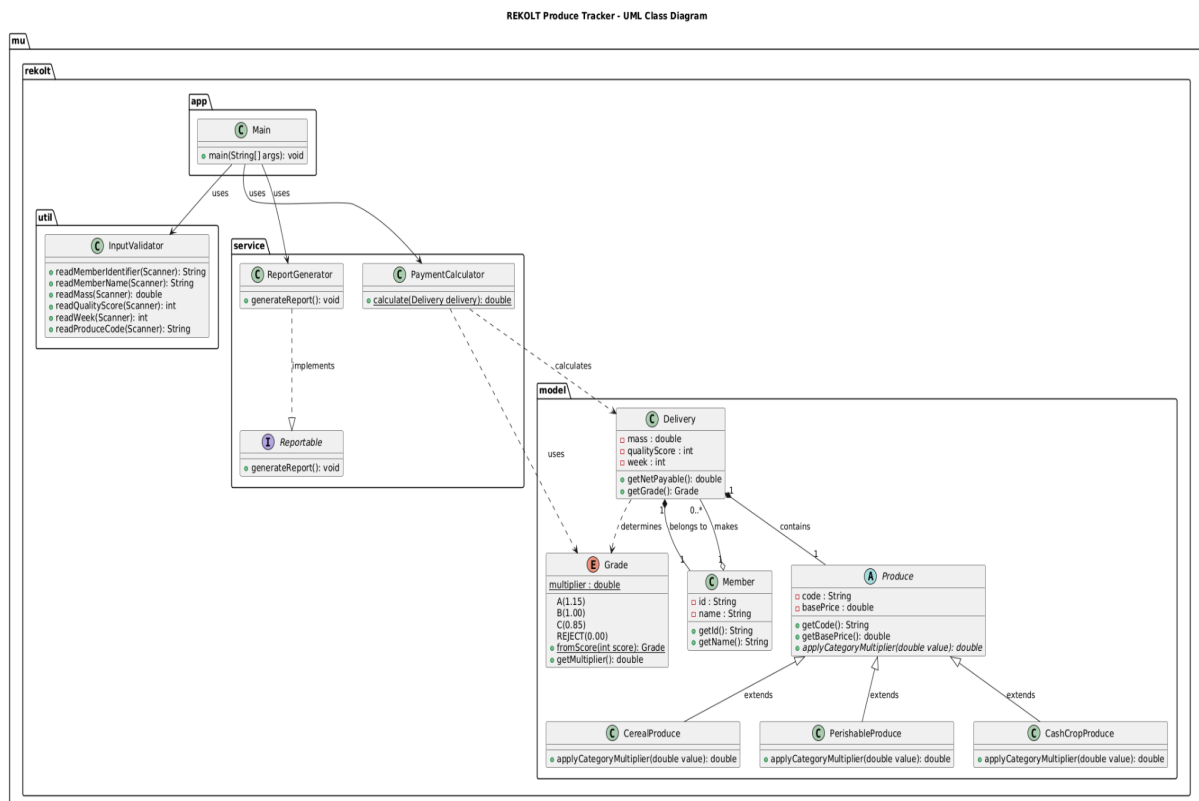
1. Scope Statement

Scope: Build a console-based Java application for the REKOLT Planters' Cooperative to record seasonal deliveries, calculate payments, and generate a season report.

Numbered Functional Requirements:

1. Record Delivery: The system shall allow a clerk to input Member ID, Name, Produce Code, Mass (kg), Quality Score (0-100), and Week (1-20).
2. Validation: The system shall reject invalid inputs (non-numeric, out of range, empty) and re-prompt the user without crashing.
3. Calculate Net Payable: The system shall calculate the Net Payable using the exact 5-step rule.
4. Handle REJECT: The system shall assign a Grade of REJECT for scores below 50, resulting in a Net Payable of 0, while still counting the mass in the weekly volume totals.
5. Aggregate Data: The system shall aggregate total payment per member, weekly volume grid (kg), and top 5 deliveries by value.
6. Generate Report: The system shall generate a single Microsoft Word document (season-report.docx) with one section per member, new pages, styled tables, and exact totals.

2. UML Class Diagram

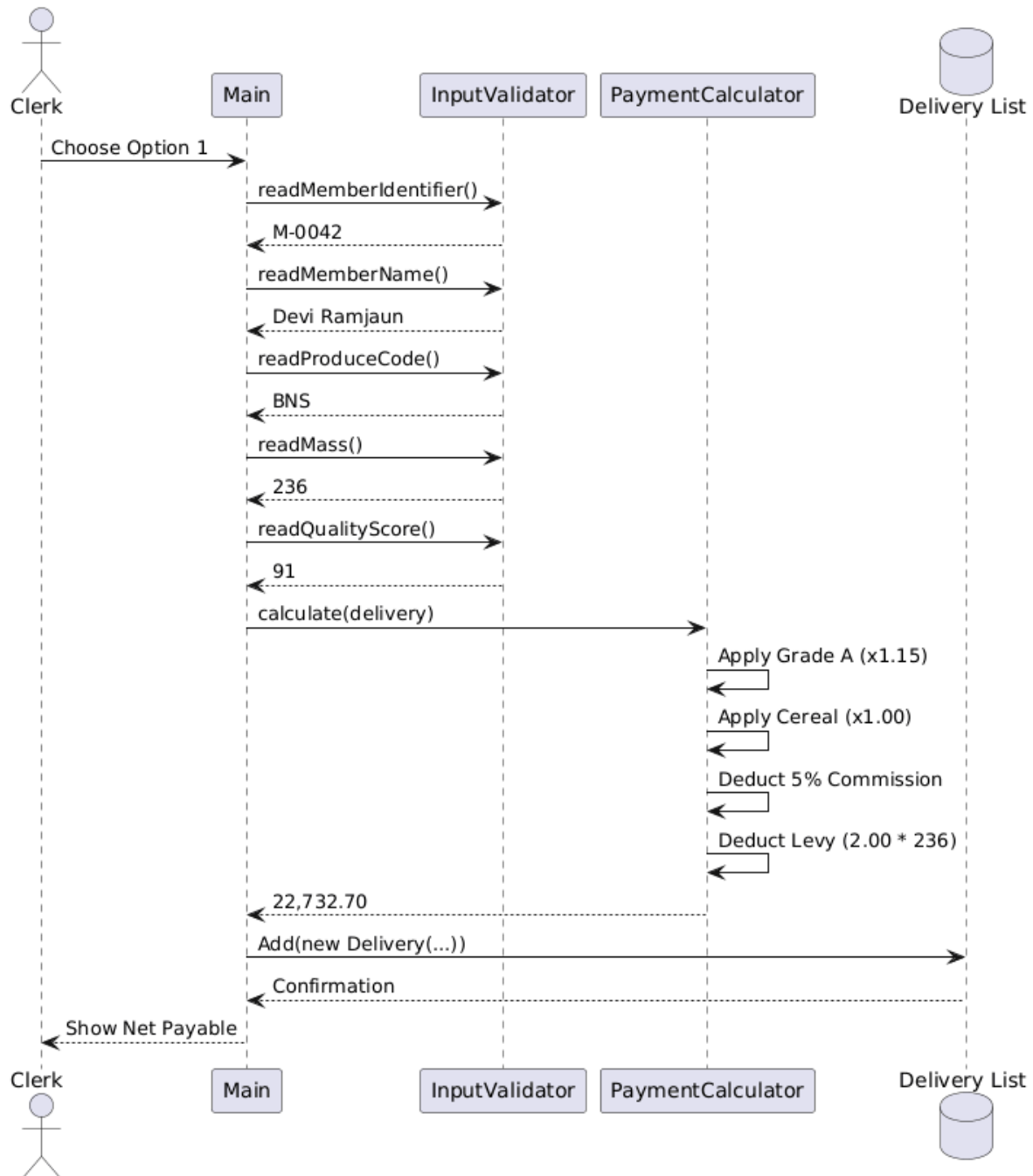


3. UML Sequence Diagram

To generate this diagram, copy this PlantUML code into the PlantUML editor, generate it, and save the image:

```
plantuml
@startuml
title REKOLT: Record a Delivery Sequence
actor Clerk
participant "Main" as Main
participant "InputValidator" as IV
participant "PaymentCalculator" as PC
database "Delivery List" as DL
Clerk -> Main: Choose Option 1
Main -> IV: readMemberIdentifier()
IV --> Main: M-0042
Main -> IV: readMemberName()
IV --> Main: Devi Ramjaun
Main -> IV: readProduceCode()
IV --> Main: BNS
Main -> IV: readMass()
IV --> Main: 236
Main -> IV: readQualityScore()
IV --> Main: 91
Main -> PC: calculate(delivery)
PC -> PC: Apply Grade A (x1.15)
PC -> PC: Apply Cereal (x1.00)
PC -> PC: Deduct 5% Commission
PC -> PC: Deduct Levy (2.00 * 236)
PC --> Main: 22,732.70
Main -> DL: Add(new Delivery(...))
DL --> Main: Confirmation
Main --> Clerk: Show Net Payable
@enduml
```

REKOLT: Record a Delivery Sequence



4. Pseudocode for Net Payment

text

FUNCTION calculateNetPayable(mass, qualityScore, produceType):

 baseValue = mass * produceType.basePrice

 grade = determineGrade(qualityScore)

 IF grade == REJECT THEN

RETURN 0.0

END IF

gradedValue = baseValue * grade.multiplier

categorizedValue = gradedValue * produceType.categoryMultiplier

commission = categorizedValue * 0.05

transportLevy = mass * 2.00

netPayable = categorizedValue - commission - transportLevy

RETURN netPayable

END FUNCTION

5. Data Dictionary

Attribute	Data	Units	Range/Format	Validation Rule
Member ID	String	N/A	M-####	Must match regex. Reject if empty.
Member Name	String	N/A	Non-empty	Reject if empty or whitespace.
Produce Code	String (Enum)	N/A	MZE, BNS, POT, TEA	Accept lower or upper case. Reject if not in list.
Mass	Double	kg	$0 < \text{mass} \leq 5000$	Reject 0, negatives, >5000, or non-numeric.
Quality Score	Integer	Points	0 to 100	Reject <0, >100, or non-whole numbers.
Week of Delivery	Integer	Weeks	1 to 20	Reject <1, >20, or non-whole numbers.
Money	Double	MUR	Varies	Keep unrounded internally. Round to 2 decimals <i>only</i> on display.

6. Rationale (Defending 3 Decisions)

Decision 1: Composition (*--) for Delivery to Produce and Member. I used Composition because a Delivery cannot logically exist without a specific Produce and Member. This fulfills Requirement 3 by ensuring every delivery is strictly bound to the correct produce type and member.

Decision 2: Abstract Class for Produce and subclasses. The rules specify distinct category multipliers (Cereal x1.00, Perishable x0.90, Cash Crop x1.10). Making Produce abstract

forces every subclass to implement `applyCategoryMultiplier()`, eliminating code duplication and preventing instantiation of a generic `Produce`.

Decision 3: `Map<String, Double>` for Total Payment per Member. The application must aggregate payments by Member ID. A Map allows for $O(1)$ lookups and aggregation by key, avoiding the need for linear searches through lists.